

如何进行词嵌入

揭秘AI大模型如何理解你的话



本次探索之旅：我们的学习路径



1. 宏观视角

AI大模型的“翻译”第一站



2. 核心部件

词元、词表与嵌入矩阵



3. 关键步骤

从文本到向量的查找过程



4. 画龙点睛

不可或缺的位置信息

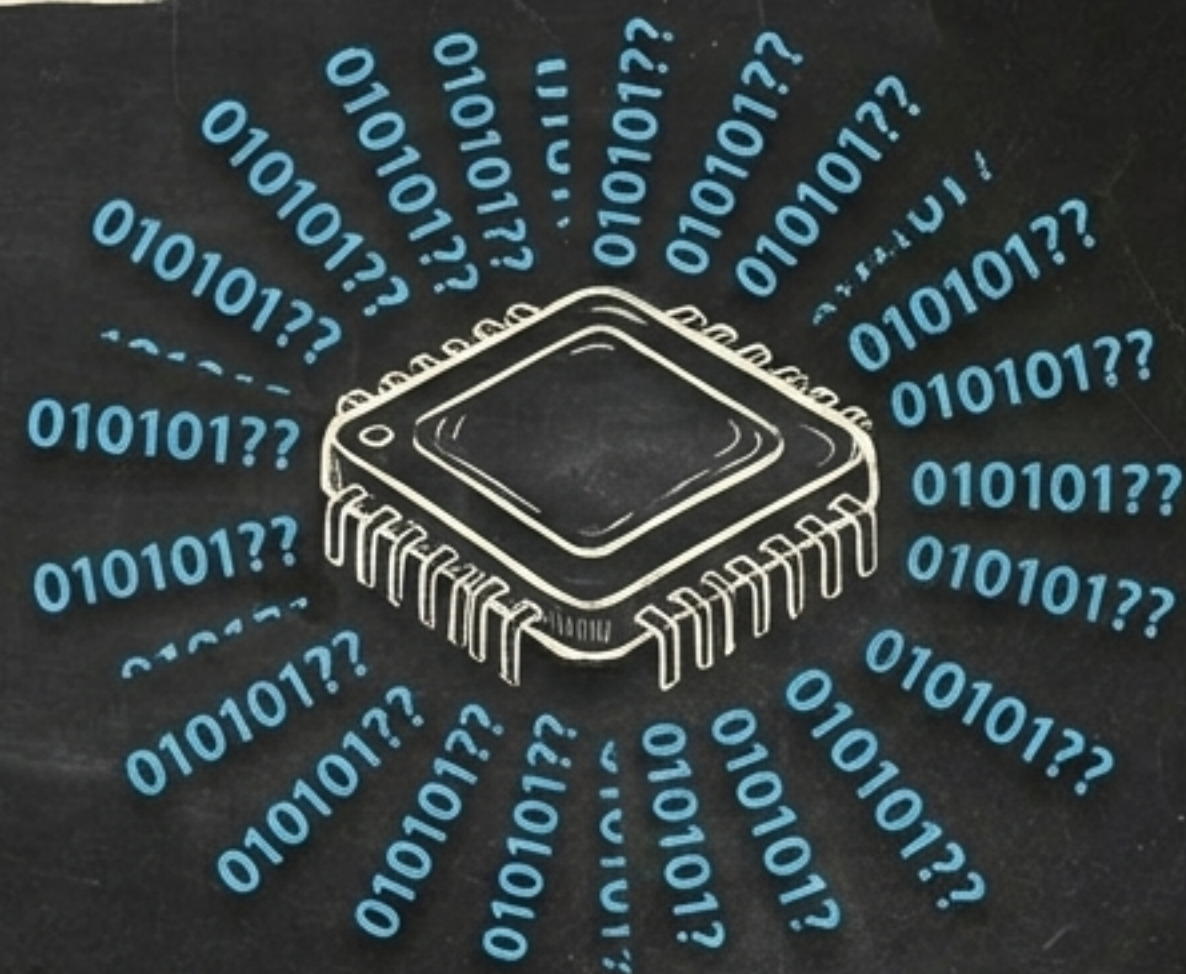
5. 融会贯通

总结与展望



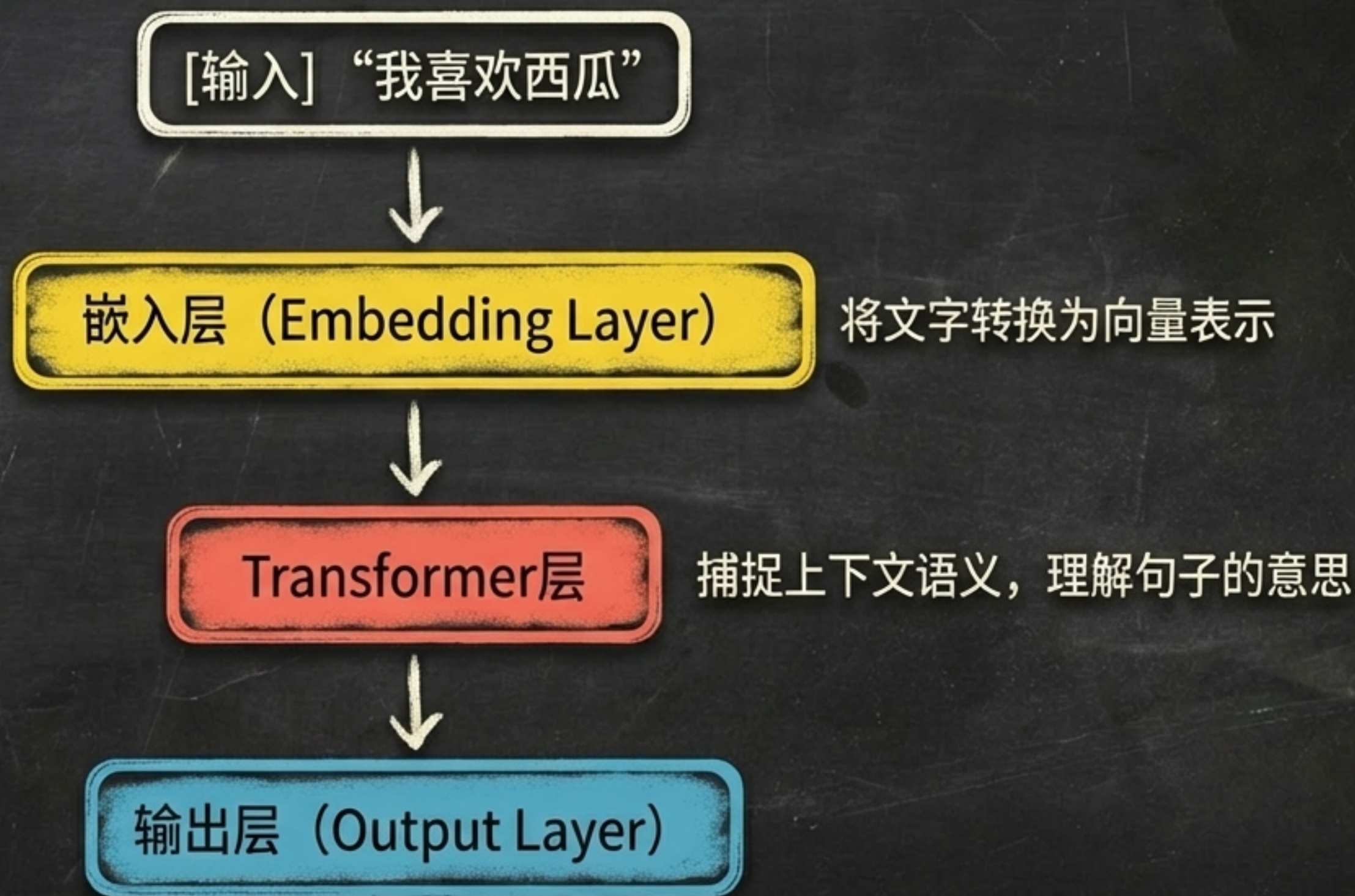
核心问题：人类语言与机器世界的鸿沟

我喜欢西瓜

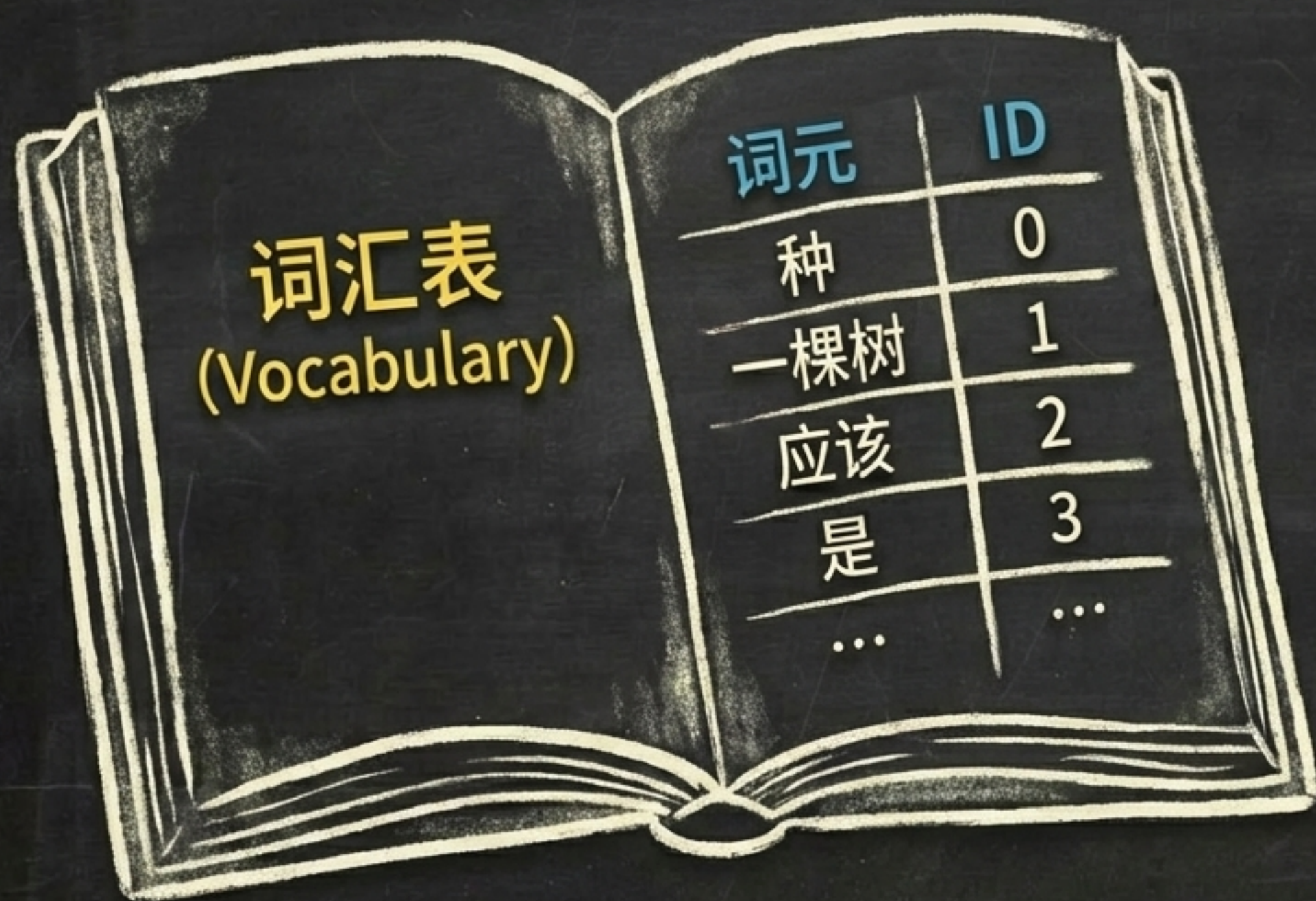


计算机的世界由数字构成，它们无法直接处理文本。因为文本不能直接进行数学计算。我们如何架起一座桥梁，让机器读懂人类的语言？

解决方案：嵌入层是模型的第一道“大门”



第一步：建立一本“词汇表”



词元	ID
种	0
一棵树	1
应该	2
是	3
...	...

模型处理的所有文本都来自于一个巨大的语料库。我们需要为语料库中的每个词元 (Token) 分配一个独一无二的“门牌号” (ID)，构建模型的“世界观”。

核心部件：神奇的嵌入矩阵

词元	ID
种	0
一棵树	1
应该是	2
是	3
...	...

词元数
(Vocabulary Size)
代表词汇表的大小

词元维度 (Embedding Dimension)

代表“定义”的丰富程度

	1	2	3	...	5	6	...	...
0.45	-0.12	0.89	0.33	...	-0.12	0.71	...	...
-0.32	0.08	0.36	0.96	...	0.55	0.59	...	...
0.45	-0.12	0.89	0.33	...	0.89	0.33	...	...
.	.	.	.	...	.	.	...	...
:	:	:	:	...	:	:	...	...
0.48	-0.14	0.97	0.17	...	0.27	0.33	...	...
.	.	.	.	...	.	.	...	...

词元的向量矩阵
(Embedding Matrix)

嵌入矩阵是模型的心脏。每一行都是一个词元的“向量定义”，用一串数字捕捉其 其复杂的语义。这个矩阵在模型训练过程中不断学习和优化。

处理流程 (1): 分词与ID化

我 喜欢 西瓜

↓ ↓ ↓

254 7561 15362

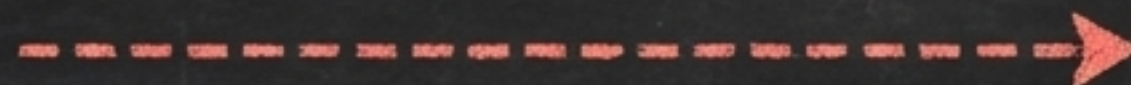
词元	ID
我	254
喜欢	7561
西瓜	15362
...	...

我们将输入句子切分成词元，然后在“词汇表”里查出它们的ID。这就像查字典的第一步：找到词条对应的页码。

处理流程 (2) : 按ID查找向量

ID for 喜欢

7561



7553	0.23	0.22	0.34	...	5.43
7564	0.68	0.80	0.23	...	0.88
⋮	⋮	⋮	⋮	⋮	⋮
7561	0.30	0.30	0.21	...	-4.92
⋮	⋮	⋮	⋮	⋮	⋮
7561	0.8	-0.03	0.03	...	3.55

7561	-0.04	0.62	0.23	...	2466
------	-------	------	------	-----	------

向量 for 喜欢

每个ID就是矩阵的行索引。我们用它精确地取出那一行——这就是该词元的向量表示。这个操作本质上是一个高效的查找。

深入原理：查找操作背后的数学

$$[0, \dots, 0, 1, 0, \dots] \times \begin{array}{|c|} \hline \text{Embedding} \\ \hline \text{Matrix} \\ \hline \end{array} = [\text{vector_7561}]$$

一个长度为词元总数，在第7561位为1，其余为0的向量

The specific row vector for '喜欢'

在数学上，这个查找操作等价于一次矩阵乘法。对于程序员来说，这可以更直观地理解为一次数组索引操作：`embedding\_matrix[7561]`。

缺失的一环：词序的重要性

狗咬人

vec(狗)

vec(咬)

vec(人)



人咬狗

vec(人)

vec(咬)

vec(狗)

仅仅查找词向量，我们只知道“有哪些词”，却丢失了它们的顺序信息。
但顺序决定了意义！模型如何区分这两个含义天差地别的句子？

解决方案：为每个位置注入“独特签名”



为了让模型理解词序，我们为句子中的每个位置（Position 0, 1, 2...）创建一个特殊的位置向量，并将其与对应的词元向量相加。

深入原理：如何计算位置向量？

位置向量的表示方法有很多，例如可以直接训练得到，但更经典的是使用固定的公式计算。

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i / d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i / d_{\text{model}})})$$

“pos”：词元在序列中的绝对位置 (0, 1, 2...)

“d_model”：模型的嵌入维度（与词元向量维度相同）

“i”：向量中的维度索引

核心思想：这种方法利用不同频率的正弦和余弦函数，为每个位置生成一个独特的、可识别的“数字指纹”。

词嵌入全流程回顾

1. 文本输入

我 喜欢 西瓜



→ 分词与ID化 (Tokenization & ID Mapping)

2. ID序列

254, 7561, 15362



→ 词向量查找 (Vector Lookup)

+ 位置向量注入 (Positional Encoding)

3. 词元向量

[vector_我]

[vector_喜欢]

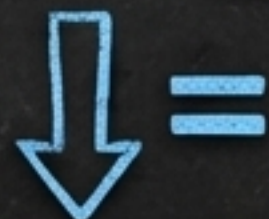
[vector_西瓜]

+

[pos_vec_0]

[pos_vec_1]

[pos_vec_2]



4. 最终输入向量

[final_vec_0]

[final_vec_1]

[final_vec_2]

深度总结：核心要点

目标 (Goal)

将离散的文本符号，转换为机器可计算的、包含语义和位置信息的连续向量。

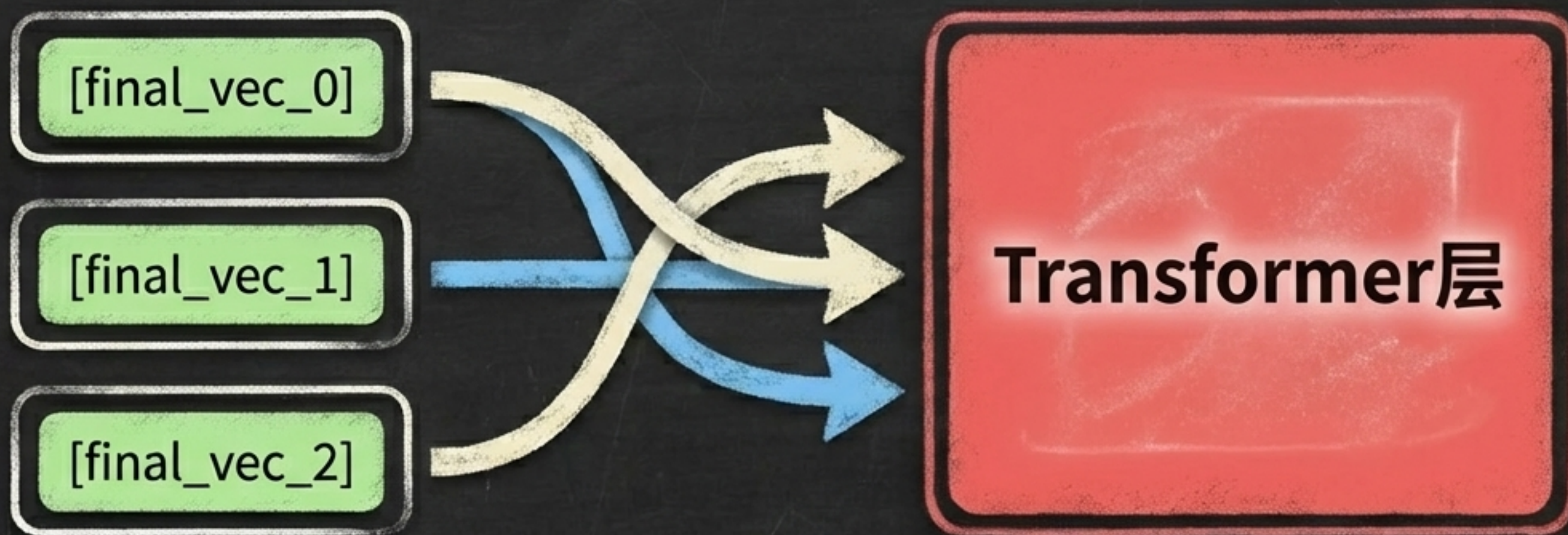
核心 (Core)

一个可训练的 **嵌入矩阵 (Embedding Matrix)** 是关键，它像一本字典，存储了每个词元的语义表示。

画龙点睛 (Finishing Touch)

位置编码 (Positional Encoding) 是确保模型理解词序的必要补充，它为向量注入了语境中至关重要的顺序信息。

旅程继续… 下一站：Transformer



这些富含信息和上下文的向量已经准备就绪，即将进入模型的“思考核心”——Transformer层。在那里，真正的上下文理解和语义推理将拉开序幕。词嵌入，正是这一切的开始。